

WEEK1 PRACTICAL PROGRAMS - 2520030487

1. Hello.c

```
#include<stdio.h>
```

```
int main()
{
    printf("Hello Linux\n");
    return 0;
}
```

Compile: gcc hello.c -o hello

./hello

Output:



```
ketana_sindhu@KetaZone:~$ gcc Hello.c -o FirstFile
ketana_sindhu@KetaZone:~$ ./FirstFile
Hello Linux
ketana_sindhu@KetaZone:~$ |
```

2. Read User Name

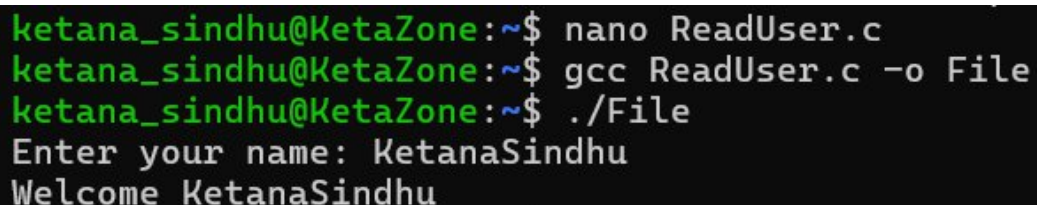
```
#include<stdio.h>
```

```
int main()
{
    char name[30];

    printf("Enter your name: ");
    scanf("%s",name);

    printf("Welcome %s\n",name);

    return 0;
}
```



```
ketana_sindhu@KetaZone:~$ nano ReadUser.c
ketana_sindhu@KetaZone:~$ gcc ReadUser.c -o File
ketana_sindhu@KetaZone:~$ ./File
Enter your name: KetanaSindhu
Welcome KetanaSindhu
```

3. Display Process ID

```
#include<stdio.h>
```

```
#include<unistd.h>
```

```

int main()
{
    printf("Process ID = %d\n",getpid());

    return 0;
}

```

```

ketana_sindhu@KetaZone:~$ nano DisplayProcessID.c
ketana_sindhu@KetaZone:~$ gcc DisplayProcessID.c -o file
ketana_sindhu@KetaZone:~$ ./file
Process ID = 1582

```

4. Fork()

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    fork();

    printf("Hello\n");

    return 0;
}

```

```

ketana_sindhu@KetaZone:~$ nano Fork.c
ketana_sindhu@KetaZone:~$ gcc Fork.c -o sindhu
ketana_sindhu@KetaZone:~$ ./sindhu
Hello
Hello

```

5. Parent and Child

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("I am Child Process\n");
    }
    else

```

```

{
    printf("I am Parent Process\n");
}

return 0;
}

```

```

ketana_sindhu@KetaZone:~$ nano ParentAndChild.c
ketana_sindhu@KetaZone:~$ gcc ParentAndChild.c -o ketana
ketana_sindhu@KetaZone:~$ ./ketana
I am Parent Process
I am Child Process

```

6. Display parent and Child PID

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child PID = %d\n",getpid());
        printf("Parent PID = %d\n",getppid());
    }
    else
    {
        printf("Parent PID = %d\n",getpid());
    }

    return 0;
}

```

```

ketana_sindhu@KetaZone:~$ nano PID.c
ketana_sindhu@KetaZone:~$ gcc PID.c -o pid
ketana_sindhu@KetaZone:~$ ./pid
Parent PID = 1759
Child PID = 1760
Parent PID = 1759

```

7. Wait()

```

#include<stdio.h>
#include<unistd.h>

```

```

#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child Running\n");
    }
    else
    {
        wait(NULL);

        printf("Parent Running After Child\n");
    }

    return 0;
}

```

```

ketana_sindhu@KetaZone:~$ nano Wait.c
ketana_sindhu@KetaZone:~$ gcc Wait.c -o wait
ketana_sindhu@KetaZone:~$ ./wait
Child Running
Parent Running After Child

```

8. Wait()

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls","ls",NULL);

    printf("After exec()\n");

    return 0;
}

```

```

ketana_sindhu@KetaZone:~$ nano wait.c
ketana_sindhu@KetaZone:~$ gcc wait.c -o WAIT
ketana_sindhu@KetaZone:~$ ./WAIT
Before exec()
DisplayProcessID.c  FirstFile  Fork.c  PID.c  ReadUser.c  WAIT  example.txt  ketana  pid  wait
File              FirstFile  Hello.c  ParentAndChild.c  S2Commands  Wait.c  file        ketana_sindhu  sindhu  wait.c

```

9. fork() + exec()

```

#include<stdio.h>

```

```
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        execlp("date","date",NULL);
    }
    else
    {
        wait(NULL);

        printf("Child Completed\n");
    }

    return 0;
}
```

```
ketana_sindhu@KetaZone:~$ nano File.c
ketana_sindhu@KetaZone:~$
ketana_sindhu@KetaZone:~$ gcc File.c -o name
ketana_sindhu@KetaZone:~$ ./name
Fri Jul 31 09:48:42 UTC 2026
Child Completed
```